



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET
KATEDRA ZA RAČUNARSTVO



Seminarski rad

Role-Based Access Control u Microsoft SQL Serveru

Student:

Jovana Stepanović br. ind. 1831

Mentor:

Prof. Aleksandar Stanimirović

Sadržaj

1. Uvod.....	3
2. Osnove RBAC modela	3
2.1. Problem koji RBAC rešava	3
2.2. Principi na kojima se RBAC zasniva	4
2.3. RBAC arhitektura u SQL Serveru	4
3. Upravljanje korisnicima i ulogama	5
3.1. Login vs User.....	5
3.2. Ugrađene role.....	7
3.3. Custom role u bankarskom scenariju	9
4. Sistem permisija, GRANT, DENY i REVOKE	13
4.1. DENY pravilo	13
4.2. Schema-level permisije.....	14
5. Unapređenja RBAC modela u SQL Server 2025	16
5.1. Contained database users	16
5.2. Dynamic Data Masking kao dopuna RBAC-u	18
6. Zaključak	20
7. Literatura	21

1. Uvod

Kontrola pristupa podacima predstavlja jedan od fundamentalnih problema u oblasti sistema za upravljanje bazama podataka. U okruženjima sa velikim brojem korisnika i različitim nivoima odgovornosti, naivni pristup (direktno dodeljivanje dozvola svakom korisniku ponaosob), pokazuje se kao neodrživ. Svaka promena u organizaciji zahteva ručno ažuriranje permisija za svakog pogođenog korisnika, greške su teško uočljive, a revizija stanja pristupa postaje praktično nemoguća. Ovaj model, pored administrativnog opterećenja, direktno narušava dva principa koja su temelj bezbednih sistema: princip minimalnih privilegija, prema kome korisnik treba da ima isključivo one dozvole koje su neophodne za obavljanje njegovog posla, i princip razdvajanja odgovornosti, prema kome nijedan korisnik ne treba da ima potpunu kontrolu nad kritičnim procesom.

Rešenje koje se nametnulo kao industrijski standard je Role-Based Access Control, to je kontrola pristupa zasnovana na ulogama, skraćeno RBAC. Umesto da se permisije dodeljuju direktno korisnicima, one se dodeljuju ulogama, a korisnici se svrstavaju u uloge koje odgovaraju njihovim radnim obavezama. Kada blagajnik postane menadžer filijale, dovoljno je promeniti ulogu i onda se sve dozvole prilagođavaju automatski.

Microsoft SQL Server podržava dodatne mehanizme poput contained database users i dynamic data masking, koji proširuju klasični RBAC model i omogućavaju finiju kontrolu nad podacima u savremenim, višekorisničkim okruženjima.

U ovom radu biće predstavljeni teorijski osnovi RBAC modela, praktična primena kroz konkretne demonstracije u SQL Server 2005, sistem permisija sa posebnim osvrtom na ponašanje DENY operatora, kao i aktuelne mogućnosti koje SQL Server 2005 donosi u ovoj oblasti.

2. Osnove RBAC modela

2.1. Problem koji RBAC rešava

U sistemima sa velikim brojem korisnika, najjednostavniji pristup kontroli pristupa podacima jeste direktno dodeljivanje permisija svakom korisniku ponaosob. Ovaj model, poznat kao **discretionary access control**, funkcioniše prihvatljivo u malim okruženjima, ali se brzo pokazuje kao neodrživ čim organizacija poraste. Kada banka ima stotine zaposlenih koji pristupaju istoj bazi podataka, administrator mora da vodi računa o permisijama svakog pojedinačnog korisnika. Svaka promena radnog mesta, svako zapošljavanje ili odlazak zaposlenog zahteva ručno ažuriranje skupa permisija, uz visok rizik od greške. Pored toga, revizija stanja (odgovor na pitanje ko tačno sme da radi šta), postaje praktično nemoguća kada su permisije rasute po stotinama individualnih korisničkih naloga.

Role-Based Access Control (RBAC), formalizovan kao standard od strane američkog Nacionalnog instituta za standarde i tehnologiju (NIST) 2000. godine, uvodi posredni nivo između korisnika i permisija u vidu uloga. Umesto da se permisije dodeljuju direktno korisnicima, one se

dodeljuju ulogama koje odgovaraju radnim pozicijama u organizaciji, a korisnici se zatim svrstavaju u odgovarajuće uloge. Kada zaposleni promeni poziciju, dovoljno je izmeniti njegovu ulogu, skup dozvola se prilagođava automatski, bez potrebe za pojedinačnim ažuriranjem permisija.

2.2. Principi na kojima se RBAC zasniva

RBAC model počiva na dva fundamentalna bezbednosna principa koja su temelj savremenih sistema za upravljanje pristupom.

Princip minimalnih privilegija nalaže da svaki korisnik treba da poseduje isključivo one permisije koje su mu neophodne za obavljanje radnih zadataka, i ništa više. U bankarskom okruženju, blagajnik koji obrađuje gotovinske transakcije ne treba da ima pristup podacima o platama zaposlenih, niti mogućnost brisanja zapisa iz baze. Svaka permisija koja prevazilazi stvarne potrebe korisnika predstavlja potencijalnu bezbednosnu ranjivost tj. bilo kroz zloupotrebu, grešku ili kompromitovanje naloga.

Princip razdvajanja odgovornosti zahteva da nijedan pojedinac ne poseduje potpunu kontrolu nad kritičnim poslovnim procesom. U finansijskim sistemima, isti korisnik ne sme da bude u poziciji da kreira transakciju i da je istovremeno odobri. Ovaj princip direktno sprečava prevaru i smanjuje rizik od greške time što kritične operacije zahtevaju učešće više različitih aktera sa različitim ulogama.

RBAC prirodno podržava oba principa: uloge se definišu prema stvarnim radnim obavezama, a raspodela uloga može biti osmišljena tako da nijedno lice ne poseduje kombinaciju rola koja bi narušila razdvajanje odgovornosti.

2.3. RBAC arhitektura u SQL Serveru

Microsoft SQL Server implementira RBAC kroz četvoroslojan hijerarhijski model koji precizno odražava strukturu organizacije koja koristi sistem.

Na vrhu hijerarhije nalazi se **Login** principal koji postoji na nivou SQL Server instance i koji predstavlja identitet koji ima pravo da se konektuje na server. Login može biti zasnovan na SQL Server autentikaciji (korisničko ime i lozinka čuvani u SQL Serveru) ili na Windows autentikaciji (Windows ili Active Directory nalog).

Na nižem nivou, unutar svake pojedinačne baze podataka, Login se mapira na **User** — principal koji postoji isključivo u kontekstu te baze i koji određuje koje objekte korisnik sme da vidi i koristi. Važno je razumeti da Login i User nisu isti entitet: Login je karika koja dozvoljava pristup serveru, dok User definiše šta se sme raditi unutar konkretne baze.

Korisnici se grupišu u **Role** koje predstavljaju imenovane skupove permisija koji odgovaraju radnim pozicijama u organizaciji. SQL Server razlikuje ugrađene role, koje dolaze uz instalaciju i

pokrivaju najčešće administrativne potrebe, i korisničke (custom) role, koje administrator kreira prema specifičnim zahtevima organizacije.

Na dnu hijerarhije nalaze se **Permission**, granularne dozvole koje definišu konkretne operacije (SELECT, INSERT, UPDATE, DELETE, EXECUTE i dr.) nad konkretnim objektima (tabelama, pogledima, procedurama, šemama). Permisije se dodeljuju ulogama, a ne direktno korisnicima, čime se postiže centralizovana i pregledna kontrola pristupa.



3. Upravljanje korisnicima i ulogama

3.1. Login vs User

Jedan od najčešćih izvora zabune pri radu sa SQL Server bezbednosnim modelom jeste razlika između logina i korisnika. Ova dva koncepta postoje na različitim nivoima sistema i služe različitim svrhama.

Login je principal na nivou SQL Server instance. On postoji u sistemskoj bazi podataka master i predstavlja identitet koji ima pravo da uspostavi konekciju na server. Login sam po sebi ne znači da korisnik ima pristup ijednoj konkretnoj bazi podataka, on samo prolazi kroz prvu kapiju.

User je principal na nivou konkretne baze podataka. On postoji unutar te baze i određuje šta korisnik sme da radi sa njenim objektima. User je gotovo uvek mapiran na postojeći Login, to mapiranje se uspostavlja pri kreiranju korisnika.

Sledeći primer demonstrira kreiranje logina na nivou servera, a zatim mapiranje tog logina na korisnika unutar baze BankDB:

SQLQuery1.s...ptimus (52))*

```

1  -- kreiranje Login-a (na nivou servera, u master bazi)
2  --CREATE LOGIN BankTellerLogin
3  --WITH PASSWORD = 'SecurePass123!';
4  --GO
5  -- login postoji u master bazi
6  SELECT name, type_desc, create_date
7  FROM sys.server_principals
8  WHERE name = 'BankTellerLogin';
9  GO
10
11  --kreiranje usera
12  --CREATE USER BankTellerUser
13  --FOR LOGIN BankTellerLogin;
14  --GO
15  -- user postoji unutar BankDB
16  SELECT name, type_desc, create_date
17  FROM sys.database_principals
18  WHERE name = 'BankTellerUser';
19  GO
20

```

100 % No issues found

Results Messages

	name	type_desc	create_date
1	BankTellerLogin	SQL_LOGIN	2026-05-28 21:13:28.707

	name	type_desc	create_date
1	BankTellerUser	SQL_USER	2026-05-28 21:13:45.400

Slika 3.1.1 — rezultati upita pokazuju da Login postoji kao SQL_LOGIN na serverskom nivou, dok odgovarajući User postoji kao SQL_USER unutar baze BankDB, čime se potvrđuje dvoslojni model autentikacije i autorizacije u SQL Serveru.

sys.server_principals je serverski pogled — prikazuje loginove i uvek vraća iste podatke bez obzira na kojoj smo bazii

sys.database_principals je pogled na nivou baze — prikazuje samo korisnike koji postoje u trenutno aktivnoj bazi

Kada se upiti izvršavaju u kontekstu master baze, sys.database_principals ne pronalazi korisnika BankTellerUser, jer taj korisnik postoji isključivo unutar BankDB. Prelaskom na BankDB, oba upita vraćaju odgovarajuće rezultate, što potvrđuje da Login i User postoje na različitim nivoima sistema.

Važna praktična napomena: ukoliko se Login obriše dok odgovarajući User i dalje postoji u bazi, taj korisnik postaje *orphaned user*, korisnik bez aktivnog logina. Prijava na server više nije moguća, ali zapis u bazi ostaje. Ovaj problem najčešće se javlja pri migraciji ili restore-u baze na drugi server, gde se SID vrednosti loginova ne poklapaju sa SID vrednostima korisnika u bazi. Orphaned usere treba redovno detektovati i uklanjati kao deo bezbednosne održavanja sistema.

3.2. Ugrađene role

SQL Server dolazi sa dva skupa unapred definisanih uloga: serverskim rolama i rolama na nivou baze podataka.

Serverske role upravljaju pristupom na nivou cele instance. Najvažnije su:

Uloga	Šta sme
sysadmin	Potpuna kontrola nad serverom, sve je dozvoljeno
securityadmin	Upravljanje loginovima i permisijama
dbcreator	Kreiranje, izmena i brisanje baza podataka
bulkadmin	Izvršavanje BULK INSERT operacija
public	Minimalne permisije, član je svaki login automatski

Članstvo u sysadmin roli treba svesti na apsolutni minimum i dodeliti isključivo DBA administratorima. U finansijskim sistemima ovo je posebno kritično, **sysadmin** može da zaobiđe svaki DENY, čak i na nivou baze.

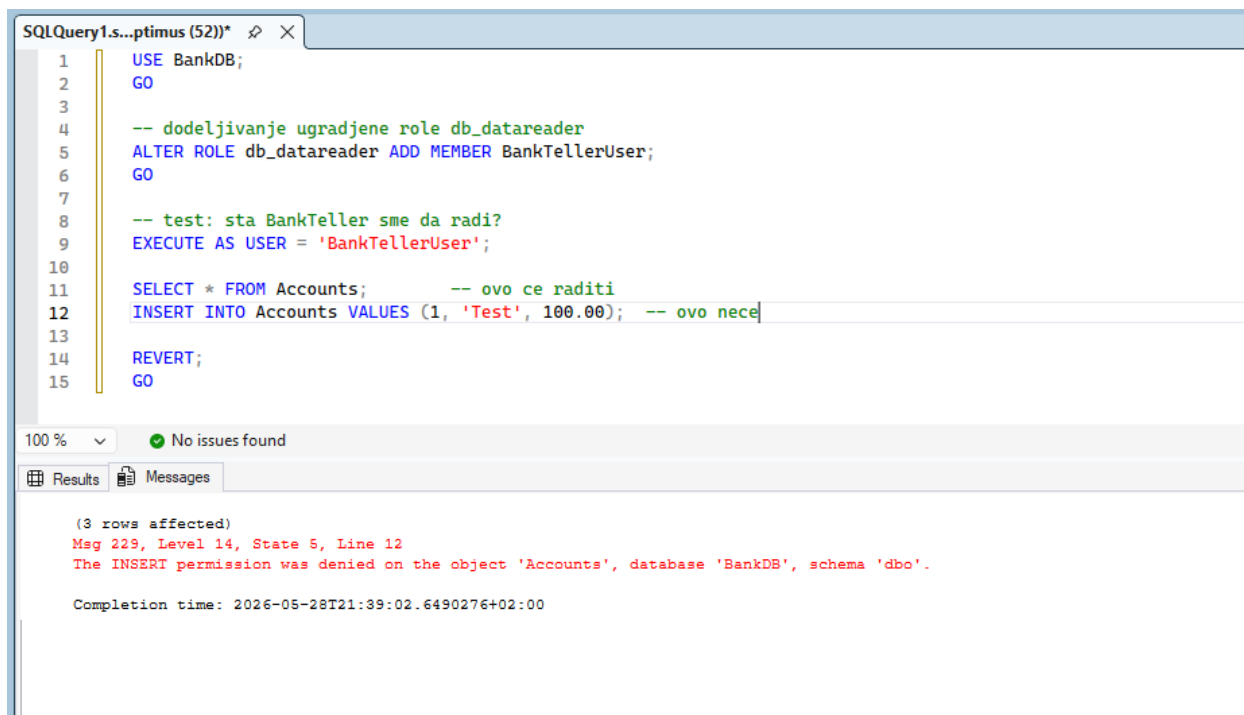
Role na nivou baze podataka upravljaju pristupom unutar konkretne baze. Najčešće korišćene su:

Uloga	Šta sme
db_datareader	SELECT na svim tabelama u bazi
db_datawriter	INSERT, UPDATE, DELETE na svim tabelama
db_ddladmin	Kreiranje i izmena strukturnih objekata (tabele, indeksi...)
db_securityadmin	Upravljanje rolama i permisijama unutar baze
db_owner	Potpuna kontrola nad bazom
db_denydatareader	Eksplicitna zabrana SELECT-a na svim tabelama
db_denydatawriter	Eksplicitna zabrana izmene podataka

Ograničenje ugrađenih rola

Ugrađene role su praktične za brzo postavljanje pristupa, ali u produkcijskim okruženjima imaju značajno ograničenje a to je da one deluju na nivou cele baze, bez mogućnosti preciziranja. Dodavanjem korisnika u **db_datareader** taj korisnik dobija SELECT na *svim* tabelama uključujući i one kojima ne bi trebalo da ima pristup. U bankarskom scenariju, blagajnik koji dobije **db_datareader** automatski vidi i tabelu Employees sa platama zaposlenih, što direktno krši princip minimalnih privilegija.

Upravo zbog ovog ograničenja produkcijska okruženja gotovo uvek kombinuju ugrađene role sa custom rolama ili ih u potpunosti zamenjuju precizno definisanim korisničkim ulogama, što je tema sledećeg poglavlja.



Slika 3.2.1 – korisnik BankTellerUser, kao član db_datareader role, uspešno izvršava SELECT upit i dobija 3 reda rezultata. Pokušaj INSERT operacije rezultuje greškom "The INSERT permission was denied", čime se potvrđuje da db_datareader dodeljuje isključivo pravo čitanja podataka.

3.3. Custom role u bankarskom scenariju

Ugrađene role pokrivaju opšte administrativne scenarije, ali u realnim poslovnim okruženjima gotovo nikada nisu dovoljne. Razlog je što one operišu na nivou cele baze, bez mogućnosti da se pristup ograniči na određene tabele ili operacije. Kada organizacija ima različite kategorije zaposlenih sa precizno definisanim radnim obavezama, potreban je finiji mehanizam kontrole. Tu dolaze **custom role** koje predstavljaju korisničke uloge koje administrator kreira i prilagođava prema stvarnim potrebama organizacije.

Kreiranje custom role u SQL Serveru svodi se na tri koraka:

1. Kreiranje uloge naredbom CREATE ROLE
2. Dodeljivanje permisija ulozi naredbama GRANT, DENY ili REVOKE
3. Dodavanje korisnika u ulogu naredbom ALTER ROLE

Ključna prednost ovog pristupa je što se permisije dodeljuju ulozi, a ne korisniku direktno. Kada novi zaposleni počne da radi kao blagajnik, administrator samo dodeli ulogu BankTeller i korisnik automatski dobija sve permisije koje ta uloga nosi, bez potrebe za ručnim definisanjem individualnih dozvola.

U nastavku je prikazan scenario sa dve uloge koje odražavaju stvarne pozicije u banci:

BankTeller (blagajnik):

- Sme da čita podatke o računima i transakcijama
- Ne sme da menja podatke
- Ne sme da pristupa podacima o zaposlenima

BranchManager (menadžer filijale) može sve što sme blagajnik, plus:

- Sme da ažurira stanja računa
- Sme da dodaje nove transakcije
- Sme da čita podatke o zaposlenima

Ovakva podela uloga direktno implementira: princip minimalnih privilegija (svako ima samo ono što mu treba) i princip razdvajanja odgovornosti (blagajnik ne može sam da kreira i odobri transakciju).

Imamo tabele Transactions i Employees gde smo kreirali custom role i dodeljujemo permisije. Kreiranje BankTeller role:

```
CREATE ROLE BankTeller;  
GO  
  
GRANT SELECT ON Accounts TO BankTeller;  
GRANT SELECT ON Transactions TO BankTeller;  
-- bez pristupa Employees tabeli!  
GO
```

Kreiranje BranchManager role:

```
CREATE ROLE BranchManager;  
GO  
  
GRANT SELECT, UPDATE ON Accounts TO BranchManager;  
GRANT SELECT, INSERT, UPDATE ON Transactions TO BranchManager;  
GRANT SELECT ON Employees TO BranchManager;  
GO
```

Kreiranje drugog Login-a i User-a za menadžera:

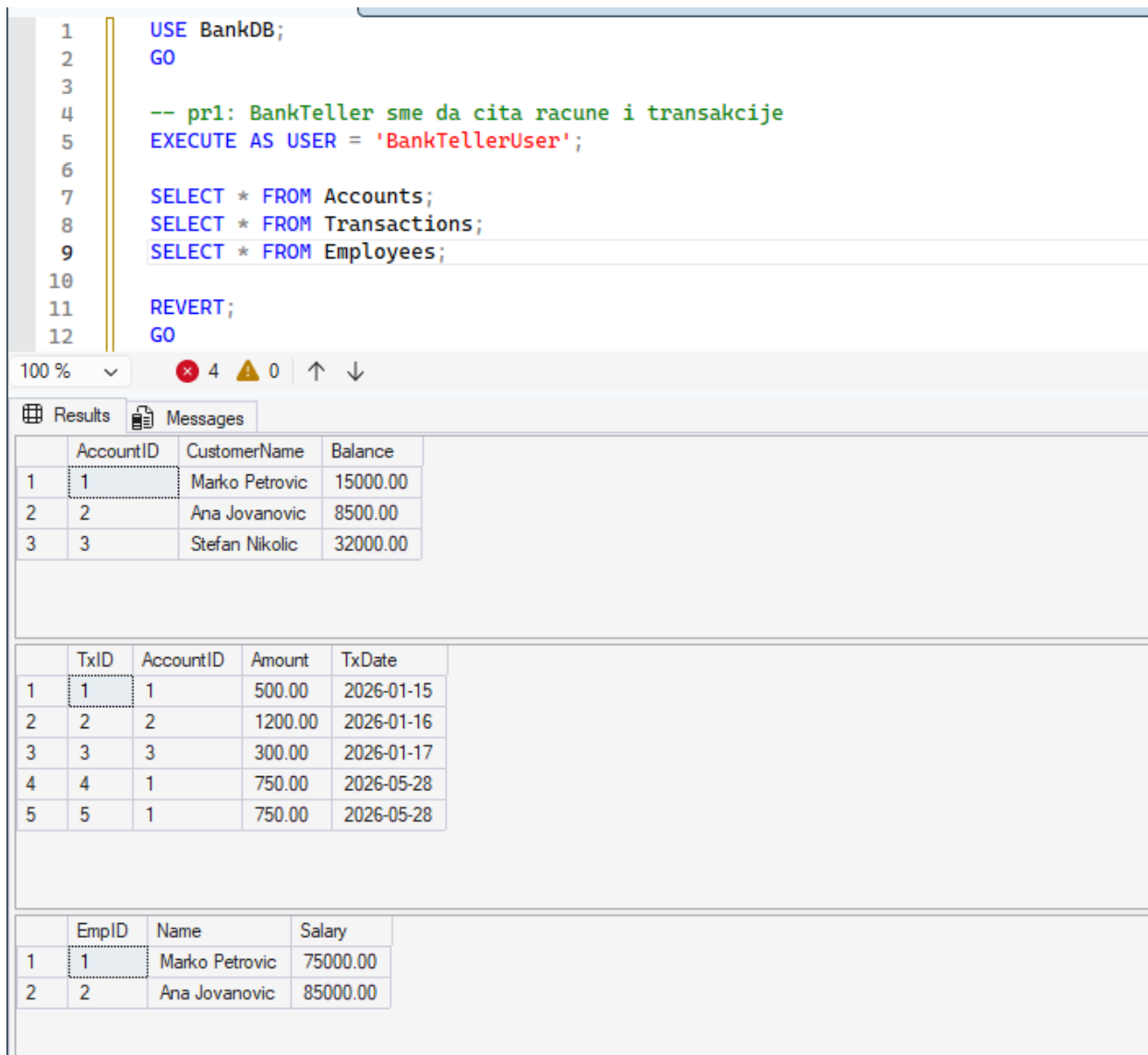
```
CREATE LOGIN BranchManagerLogin WITH PASSWORD = 'ManagerPass456!';  
GO  
  
CREATE USER BranchManagerUser FOR LOGIN BranchManagerLogin;  
GO
```

Dodeljivanje uloga korisnicima:

```
ALTER ROLE BankTeller ADD MEMBER BankTellerUser;  
ALTER ROLE BranchManager ADD MEMBER BranchManagerUser;
```

GO

Nakon kreiranja custom rola i dodeljivanja permisija, pristupamo testiranju. Prilikom prvog testiranja BankTellerUser je i dalje bio član ugrađene db_datareader role iz prethodnog poglavlja. Kako SQL Server akumulira permisije iz svih rola kojima korisnik pripada, BankTellerUser je dobijao SELECT na svim tabelama kroz db_datareader, uključujući i Employees tabelu kojoj po definiciji BankTeller role ne bi trebalo da ima pristup. Ovo ponašanje prikazano je na Slici 3.3.1, gde results tab pokazuje da su sva tri SELECT upita uspešno vratila podatke, uključujući i plate zaposlenih.



```
1  USE BankDB;
2  GO
3
4  -- pr1: BankTeller sme da cita racune i transakcije
5  EXECUTE AS USER = 'BankTellerUser';
6
7  SELECT * FROM Accounts;
8  SELECT * FROM Transactions;
9  SELECT * FROM Employees;
10
11 REVERT;
12 GO
```

100 % 4 0

Results Messages

	AccountID	CustomerName	Balance
1	1	Marko Petrovic	15000.00
2	2	Ana Jovanovic	8500.00
3	3	Stefan Nikolic	32000.00

	TxID	AccountID	Amount	TxDate
1	1	1	500.00	2026-01-15
2	2	2	1200.00	2026-01-16
3	3	3	300.00	2026-01-17
4	4	1	750.00	2026-05-28
5	5	1	750.00	2026-05-28

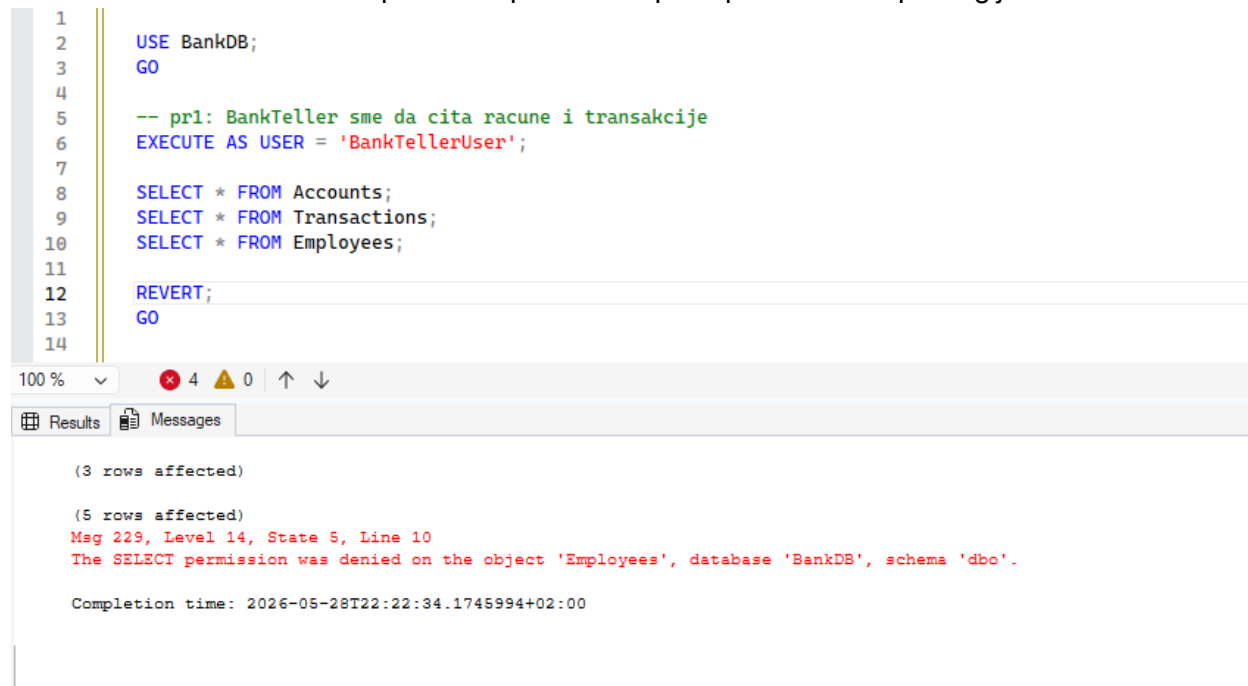
	EmpID	Name	Salary
1	1	Marko Petrovic	75000.00
2	2	Ana Jovanovic	85000.00

Slika 3.3.1. – BankTellerUser uspešno čita sve tri tabele, uključujući Employees, zbog akumulacije permisija iz db_datareader role.

Ovaj rezultat direktno demonstrira ograničenje mešanja ugrađenih i custom rola, db_datareader efektivno poništava precizno definisane restrikcije custom role. Rešenje je uklanjanje korisnika iz db_datareader:

```
ALTER ROLE db_datareader DROP MEMBER BankTellerUser; GO
```

Nakon uklanjanja, testiranje daje očekivano ponašanje prikazano na Slici 3.3.1.1. Messages tab pokazuje da su upiti nad Accounts i Transactions uspešno izvršeni (3 i 5 redova), dok pokušaj pristupa tabeli Employees rezultuje greškom The SELECT permission was denied, što potvrđuje da custom BankTeller rola ispravno implementira princip minimalnih privilegija.



```
1
2  USE BankDB;
3  GO
4
5  -- pr1: BankTeller sme da cita racune i transakcije
6  EXECUTE AS USER = 'BankTellerUser';
7
8  SELECT * FROM Accounts;
9  SELECT * FROM Transactions;
10 SELECT * FROM Employees;
11
12 REVERT;
13 GO
14
```

100 % 4 0

Results Messages

(3 rows affected)

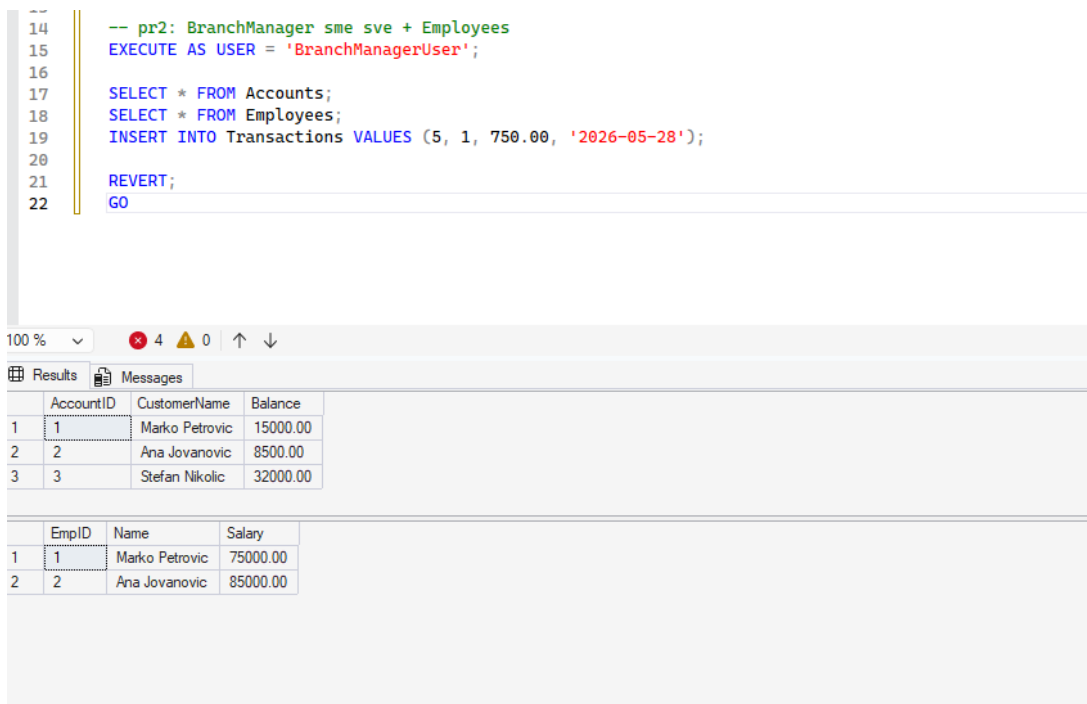
(5 rows affected)

Msg 229, Level 14, State 5, Line 10
The SELECT permission was denied on the object 'Employees', database 'BankDB', schema 'dbo'.

Completion time: 2026-05-28T22:22:34.1745994+02:00

Slika 3.3.1.1. – nakon uklanjanja iz db_datareader, BankTellerUser uspešno čita Accounts i Transactions, ali dobija grešku pri pristupu tabeli Employees.

Što se tiče BranchManagerUser, kao član BranchManager role, uspešno čita tabele Accounts i Employees, te uspešno upisuje novu transakciju u tabelu Transactions, čime se potvrđuje da BranchManager role ispravno implementira viši nivo pristupa u odnosu na BankTeller.



Slika 3.3.2 – BranchManagerUser uspešno pristupa tabelama Accounts i Employees i dodaje novu transakciju

4. Sistem permisija, GRANT, DENY i REVOKE

SQL Server kontroliše pristup objektima kao što su tabele, pogledi, procedure, funkcije i šeme, kroz tri osnovne naredbe:

- GRANT dodeljuje permisiju korisniku ili ulozi.
- REVOKE uklanja prethodno dodeljenu permisiju i vraća korisnika u neutralno stanje, ne garantuje zabranu pristupa ukoliko korisnik tu permisiju ima kroz neku drugu ulogu.
- DENY eksplicitno zabranjuje permisiju i ima prioritet nad svim GRANT operacijama.

4.1. DENY pravilo

DENY ima prioritet nad svim GRANT operacijama. Ovo pravilo važi bez izuzetka, bez obzira na broj uloga kojima korisnik pripada i koliko njih mu eksplicitno dodeljuje tu permisiju, jedan DENY je dovoljan da je blokira u potpunosti.

Ovo pravilo ima direktnu praktičnu primenu u bankarskim i finansijskim sistemima. Zamislimo situaciju u kojoj banka uvodi privremenu restrikciju, određenoj grupi korisnika treba onemogućiti pristup osetljivim podacima tokom revizije. Umesto da se menjaju sve uloge kojima ti korisnici pripadaju, administrator postavlja jedan DENY i pristup je blokiran odmah, centralizovano.

Važno je razumeti i odnos između DENY i REVOKE. REVOKE uklanja permisiju ali ne garantuje zabranu, dok DENY aktivno postavlja zabranu koja ne može biti nadjačana nijednom drugom permisijom. Zbog toga se u produkcijskim okruženjima DENY koristi kada je potrebno garantovati da određeni korisnik nikada ne sme da pristupi određenom objektu, bez obzira na buduće promene u članstvu uloga.

Da bismo demonstrirali ovo ponašanje, kreiramo ulogu RestrictedUsers sa DENY SELECT na tabeli Accounts i dodeljujemo je korisniku BankTellerUser koji već ima GRANT SELECT kroz BankTeller ulogu:

```
SQLQuery1.s...timus (120))* X
1  USE BankDB;
2  GO
3
4  -- kreiramo ulogu koja zabranjuje pristup
5  CREATE ROLE RestrictedUsers;
6  GO
7
8  DENY SELECT ON Accounts TO RestrictedUsers;
9  GO
10
11 -- dodajemo BankTellerUser u obe uloge
12 -- BankTeller ima GRANT SELECT na Accounts
13 -- RestrictedUsers ima DENY SELECT na Accounts
14 ALTER ROLE RestrictedUsers ADD MEMBER BankTellerUser;
15 GO
16
17 -- primer:
18 EXECUTE AS USER = 'BankTellerUser';
19
20 SELECT * FROM Accounts;
21
22 REVERT;
23 GO
```

100 % No issues found

Messages

Msg 229, Level 14, State 5, Line 20
The SELECT permission was denied on the object 'Accounts', database 'BankDB', schema 'dbo'.

Completion time: 2026-05-30T09:58:13.7525089+02:00

Slika 4.1.1. – iako BankTellerUser ima GRANT SELECT kroz BankTeller ulogu, jedan DENY iz RestrictedUsers uloge blokira pristup u potpunosti.

4.2. Schema-level permisije

Do sada smo permisije dodavali na nivou pojedinačnih tabela, posebno za Accounts, posebno za Transactions, posebno za Employees. Ovaj pristup, poznat kao **object-level** dodeljivanje permisija, funkcioniše prihvatljivo kada baza ima mali broj objekata. Međutim, u realnim sistemima baza može imati desetine ili stotine tabela i ručno dodeljivanje permisije za svaku od njih postaje nepraktično i sklono greškama.

Rešenje je dodeljivanje permisija na nivou **šeme**. Šema je logički kontejner koji grupiše srodne objekte unutar baze. SQL Server po defaultu koristi šemu dbo (database owner) u kojoj se nalaze svi objekti koje smo kreirali: Accounts, Transactions i Employees su sve deo dbo šeme.

Kada se permisija dodeli na nivou šeme, ona automatski važi za **sve postojeće i sve buduće objekte** unutar te šeme. Ako banka sutra doda novu tabelu LoanApplications u dbo šemu, korisnici koji imaju schema-level permisiju automatski dobijaju pristup toj tabeli bez ikakve dodatne intervencije administratora.

U kompleksnijim sistemima šeme se koriste i za organizaciju objekata po poslovnim domenima. Banka može imati šemu Accounts za sve objekte vezane za račune, šemu HR za kadrovske podatke i šemu Reporting za izveštaje. Ovakva organizacija omogućava preciznu kontrolu pristupa, uloga BankTeller dobija SELECT na šemi Accounts, ali nema nikakav pristup šemi HR.

Da bismo demonstrirali schema-level pristup, kreiramo ulogu SchemaReader i dodeljujemo joj SELECT na celoj dbo šemi jednom naredbom. Umesto tri posebna GRANT-a za svaku tabelu, jedan GRANT pokriva sve postojeće i buduće objekte unutar šeme:

```
GRANT SELECT ON SCHEMA::dbo TO SchemaReader;
```

Ovo pokriva Accounts, Transactions, Employees i sve buduće tabele.

SQLQuery1.s...timus (120))*

```

12
13 -- kreiramo novog korisnika
14 CREATE LOGIN SchemaReaderLogin WITH PASSWORD = 'SchemaPass789!';
15 GO
16
17 CREATE USER SchemaReaderUser FOR LOGIN SchemaReaderLogin;
18 GO
19
20 ALTER ROLE SchemaReader ADD MEMBER SchemaReaderUser;
21 GO
22
23 -- primer: SchemaReaderUser sme da cita sve tabele u dbo semi
24 EXECUTE AS USER = 'SchemaReaderUser';
25
26 SELECT * FROM Accounts;
27 SELECT * FROM Transactions;
28 SELECT * FROM Employees;
29
30 REVERT;
31 GO

```

100 % No issues found

Results Messages

	AccountID	CustomerName	Balance
1	1	Marko Petrovic	15000.00
2	2	Ana Jovanovic	8500.00
3	3	Stefan Nikolic	32000.00

	TxID	AccountID	Amount	TxDate
1	1	1	500.00	2026-01-15
2	2	2	1200.00	2026-01-16
3	3	3	300.00	2026-01-17
4	4	1	750.00	2026-05-28
5	5	1	750.00	2026-05-28

	EmpID	Name	Salary
1	1	Marko Petrovic	75000.00
2	2	Ana Jovanovic	85000.00

Slika 4.2.1 – SchemaReaderUser uspešno čita sve tri tabele kroz jedan schema-level GRANT na dbo šemi, bez dodatnih object-level permisija.

5. Unapređenja RBAC modela u SQL Server 2025

5.1. Contained database users

U standardnom SQL Server modelu koji smo koristili kroz ceo rad, svaki korisnik mora imati Login na nivou servera koji se mapira na User unutar baze. Ova zavisnost između logina i korisnika, iako logična, u određenim scenarijima predstavlja praktičan problem.

Zamislamo situaciju u kojoj se BankDB premešta na drugi server, kroz backup/restore ili Always On failover. Svi loginovi ostaju na starom serveru, dok korisnici u bazi dolaze na novi server bez odgovarajućih loginova. Rezultat su orphaned useri koje smo pominjali u poglavlju 3.1 to su korisnici koji postoje u bazi ali ne mogu da se prijave.

Contained database users rešavaju ovaj problem eliminisanjem zavisnosti od server-level logina. Korisnik i njegova lozinka čuvaju se direktno unutar baze podataka, a ne u master bazi. To znači da baza postaje **prenosiva**, može se premestiti na bilo koji SQL Server i korisnici će moći da se prijave bez ikakvih dodatnih podešavanja.

Ovo ima direktnu primenu u bankarskim sistemima koji koriste visoku dostupnost, pri failoveru na rezervni server korisnici nastavljaju da rade bez prekida i bez intervencije administratora.

Međutim, contained users nose i jedan bezbednosni rizik koji je važno napomenuti: korisnik sa ALTER ANY USER pravom unutar baze može da kreira contained usera i time efektivno zaobiđe centralnu kontrolu loginova koju vrši sysadmin. Zbog toga se preporučuje pažljivo dodeljivanje ALTER ANY USER permisije.

Da bismo demonstrirali contained usera, prvo omogućavamo contained authentication na serveru i postavljamo BankDB kao partially contained bazu. Nakon toga kreiramo korisnika direktno unutar baze bez ikakvog server-level logina:

Omogućavamo contained database authentication na serveru

```
EXEC sp_configure 'contained database authentication', 1;  
RECONFIGURE;  
GO
```

Postavljamo BankDB kao partially contained bazu

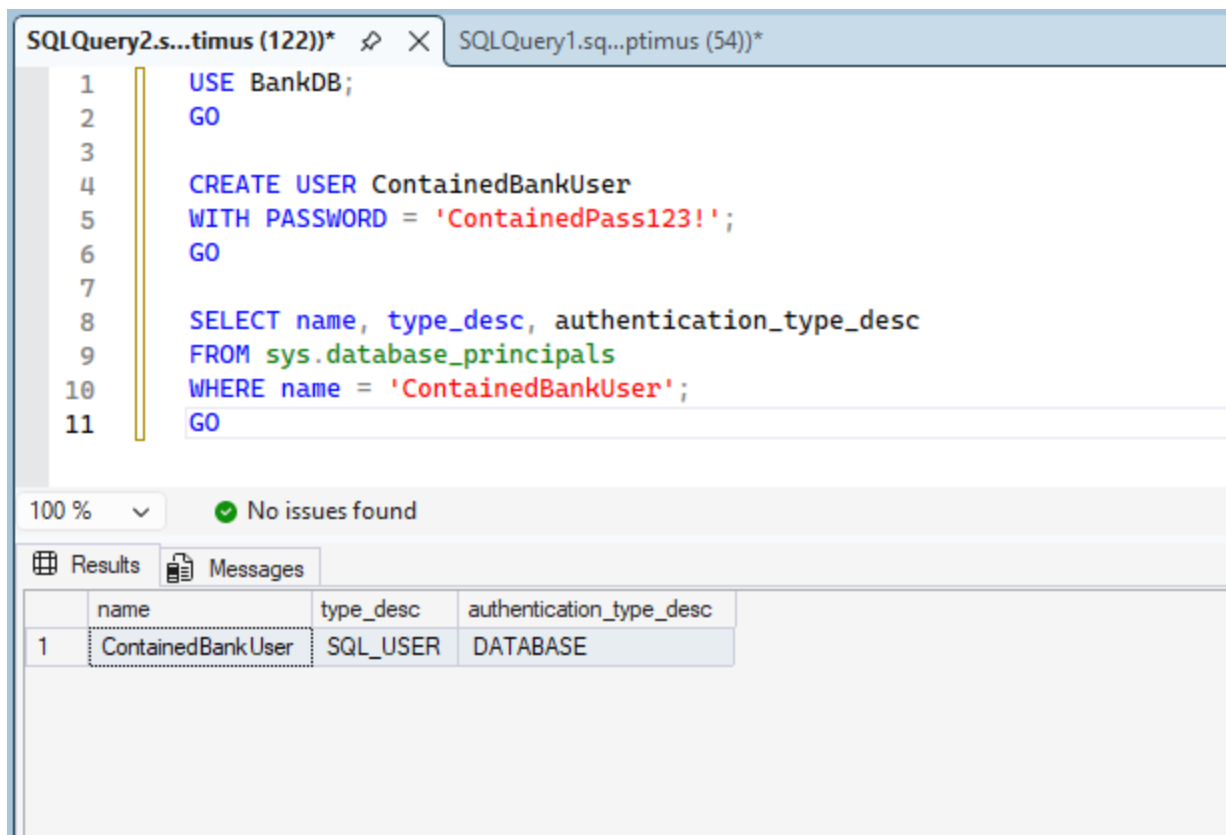
```
ALTER DATABASE BankDB SET CONTAINMENT = PARTIAL;  
GO
```

Kreiramo contained usera direktno u bazi, bez ikakvog Login-a na nivou servera

```
USE BankDB;  
GO  
CREATE USER ContainedBankUser  
WITH PASSWORD = 'ContainedPass123!';  
GO
```

Provera: contained user postoji u bazi

```
SELECT name, type_desc, authentication_type_desc  
FROM sys.database_principals  
WHERE name = 'ContainedBankUser';  
GO
```



Slika 5.1.1 — Kolona `authentication_type_desc` prikazuje vrednost `DATABASE`, što potvrđuje da `ContainedBankUser` postoji isključivo unutar `BankDB` i ne zavisi od server-level logina.

5.2. Dynamic Data Masking kao dopuna RBAC-u

RBAC model koji smo implementirali kroz prethodna poglavlja kontroliše **ko sme da pristupi kojim objektima**. Međutim, postoji scenarijo koji RBAC sam po sebi ne pokriva elegantno a to je situacija kada korisnik treba da ima pristup tabeli, ali ne sme da vidi određene osetljive podatke unutar te tabele.

Zamislimo blagajnika koji mora da vidi tabelu `Accounts` kako bi mogao da radi sa klijentima, ali ne sme da vidi tačno stanje računa. Ili korisničku podršku koja vidi ime klijenta ali ne sme da vidi broj kartice. RBAC može da zabrani pristup celoj tabeli, ali ne može elegantno da sakrije samo određene kolone za određene uloge.

Dynamic Data Masking (DDM) rešava ovaj problem. DDM automatski maskuje podatke u kolonama i korisnik koji nema pravo da vidi osetljive podatke dobija maskiran rezultat umesto stvarnih vrednosti, bez ikakve promene u aplikaciji ili upitu. Korisnik i dalje može da izvršava `SELECT`, ali vidi npr. `XXXX` umesto broja kartice ili `0.00` umesto stvarnog stanja računa.

SQL Server podržava nekoliko tipova maskiranja::

- **default** — prikazuje 0 za brojeve, `XXXX` za tekst, `01-01-1900` za datume

- **email** — prikazuje samo prvo slovo i domen, npr. a@XXXX.com
- **partial** — prikazuje definisani prefiks i sufiks, ostatak maskuje
- **random** — za numeričke vrednosti, prikazuje nasumičan broj u zadatom opsegu

Važno je napomenuti da DDM nije zamena za RBAC, korisnik koji ima UNMASK permisiju ili dovoljno visoke privilegije može da vidi prave vrednosti. DDM je dopuna RBAC modelu koja dodaje još jedan sloj zaštite osetljivih podataka.

Dodajemo masking na Balance kolonu u Accounts tabeli

```
ALTER TABLE Accounts
ALTER COLUMN Balance ADD MASKED WITH (FUNCTION = 'default()');
GO
```

Dodajemo masking na Salary kolonu u Employees tabeli

```
ALTER TABLE Employees
ALTER COLUMN Salary ADD MASKED WITH (FUNCTION = 'default()');
GO
```

Kreiramo korisnika koji nema UNMASK permisiju

```
CREATE USER MaskedUser WITHOUT LOGIN;
GRANT SELECT ON Accounts TO MaskedUser;
GRANT SELECT ON Employees TO MaskedUser;
```

Nakon postavljanja maski i kreiranja korisnika, testiramo ponašanje izvršavanjem istog upita kao MaskedUser i kao administrator:

```

19
20      -- pr1: MaskedUser vidi maskovane podatke
21      EXECUTE AS USER = 'MaskedUser';
22      SELECT * FROM Accounts;
23      SELECT * FROM Employees;
24      REVERT;
25      GO
26
27      -- pr2: admin vidi prave podatke
28      SELECT * FROM Accounts;
29      SELECT * FROM Employees;
30      GO

```

100 % ✔ No issues found

Results Messages

	AccountID	CustomerName	Balance
1	1	Marko Petrovic	0.00
2	2	Ana Jovanovic	0.00
3	3	Stefan Nikolic	0.00

	EmpID	Name	Salary
1	1	Marko Petrovic	0.00
2	2	Ana Jovanovic	0.00

	AccountID	CustomerName	Balance
1	1	Marko Petrovic	15000.00
2	2	Ana Jovanovic	8500.00
3	3	Stefan Nikolic	32000.00

	EmpID	Name	Salary
1	1	Marko Petrovic	75000.00
2	2	Ana Jovanovic	85000.00

Slika 5.2.1. – MaskedUser vidi maskirane vrednosti (0.00), dok administrator vidi prave podatke, isti upit, različit rezultat u zavisnosti od UNMASK permisije.

6. Zaključak

Ovaj rad prikazao je implementaciju Role-Based Access Control modela u Microsoft SQL Server 2025, kroz teorijsku osnovu i praktične demonstracije na primeru baze podataka BankDB.

Kroz poglavlja rada demonstrirano je kako SQL Server implementira RBAC kroz četvoroslojan model: Login, User, Role i Permission. Kako ovaj model prirodno podržava dva ključna bezbednosna principa: princip minimalnih privilegija i princip razdvajanja odgovornosti. Praktični bankarski scenario sa ulogama BankTeller i BranchManager pokazao je kako precizno definisane custom role omogućavaju kontrolu pristupa prilagođenu stvarnim poslovnim potrebama, za razliku od ugrađenih rola koje operišu na nivou cele baze.

Sistem permisija demonstriran je kroz GRANT, DENY i REVOKE operacije, sa posebnim osvrtom na DENY pravilo koje ima apsolutni prioritet nad svim GRANT operacijama, što predstavlja jedan od najvažnijih mehanizama za garantovanje bezbednosti u produkcijskim okruženjima. Schema-

level permisije pokazale su kako se upravljanje pristupom može centralizovati i automatizovati za sve buduće objekte unutar šeme.

U okviru aktuelnih mogućnosti SQL Server 2025, prikazani su contained database users koji rešavaju problem zavisnosti korisnika od server-level logina i omogućavaju prenosivost baze, kao i Dynamic Data Masking koji dopunjuje RBAC model maskiranjem osetljivih podataka na nivou kolona bez izmene aplikacijske logike.

Određene teme nisu demonstrirane u praktičnom delu rada zbog ograničenja okruženja. Integracija sa Active Directory i Microsoft Entra ID zahteva domensku infrastrukturu, dok konfiguracija visoke dostupnosti kroz Always On Availability Groups zahteva višeinstančno SQL Server okruženje. Ove teme su dokumentovane u zvaničnoj Microsoft dokumentaciji navedenoj u literaturi.

RBAC nije jednokratno podešavanje već kontinuiran proces, permisije treba redovno revidirati, orphaned usere detektovati i uklanjati, a nove objekte svesno smestiti u odgovarajuće šeme. Kombinacija ugrađenih mehanizama SQL Server 2025 sa pažljivo definisanim custom rolama predstavlja solidan temelj za bezbednost podataka u savremenim poslovnim sistemima.

7. Literatura

1. Microsoft, "Server-Level Roles", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/security/authentication-access/server-level-roles?view=sql-server-ver17>
2. Microsoft, "Database-Level Roles", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/security/authentication-access/database-level-roles?view=sql-server-ver17>
3. Microsoft, "Permissions (Database Engine)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/security/permissions-database-engine?view=sql-server-ver17>
4. Microsoft, "GRANT Object Permissions (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/t-sql/statements/grant-object-permissions-transact-sql?view=sql-server-ver17>
5. Microsoft, "REVOKE (Transact-SQL)", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/t-sql/statements/revoke-transact-sql?view=sql-server-ver17>
6. Microsoft, "Contained Database Users — Making Your Database Portable", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/relational-databases/security/contained-database-users-making-your-database-portable?view=sql-server-ver17>
7. Microsoft, "What's New in SQL Server 2025", Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/sql/sql-server/what-s-new-in-sql-server-2025?view=sql-server-ver17>